

Next Steps - Teams Bridge Implementation

Quick Start

This is the implementation roadmap for the **Teams Bridge** service using **Microsoft 365 Agents SDK**.

For complete details, see: [../docs/TEAMS_SDK_INTEGRATION.md](https://github.com/microsoft/teams-bridge/blob/main/docs/TEAMS_SDK_INTEGRATION.md)

Phase 1: Azure Setup (José Arturo - IT Admin)

Timeline: 1-2 hours

Owner: Azure/Teams Administrator

Status: ☒ COMPLETE

Completed ☒

- ☒ **Register bot in Azure AD**
 - App Name: `ftc-ispilot-corp-prod`
 - Microsoft App ID: `53ead9aa-24aa-44e7-b485-81a988e7492f`
 - Object ID: `f85873ef-78e3-4e6e-97dc-48c4f0e95139`
 - Tenant ID: `c4a8886b-f140-478b-ac47-249555c30afd`
 - Owners: *To be assigned*
- ☒ **Generate Client Secret (App Password)**
 - Secret ID: `b8130e5a-3446-49ef-8267-056e21cabec2`
 - Secret Value: ☒ Stored in `.env.example`
 - Expiration: 24 months
- ☐ **Configure Teams Bot Channel**
 - Go to Azure Bot Service
 - Create bot resource with Azure credentials above
 - Note: Messaging Endpoint will be: `https://teams-ispilot-bridge-xxxxxx.run.app/api/messages`
 - Enable Teams channel
- ☐ **Verify Azure Permissions**
 - Confirm Azure subscription has Teams integration enabled
 - Verify Microsoft Graph API permissions

Final Deliverables:

- ☒ Microsoft App ID: `53ead9aa-24aa-44e7-b485-81a988e7492f`
- ☒ Microsoft App Secret: ☒ Configured
- ☒ Bot Resource Name: `ftc-ispilot-corp-prod`

- ☒ Object ID: `f85873ef-78e3-4e6e-97dc-48c4f0e95139`
 - ☒ Tenant ID: `c4a8886b-f140-478b-ac47-249555c30afd`
 - ☒ Secret ID: `b8130e5a-3446-49ef-8267-056e21cabec2`
-

Phase 2: Build Teams Bridge (Development)

Timeline: 4-6 hours

Owner: Development Team

Prepare

- ☐ Review [README.md](#) - Architecture and local setup
- ☐ Review [../docs/TEAMS_SDK_INTEGRATION.md](#) - Complete guide
- ☐ Have Azure credentials from Phase 1 ready

Implementation

1. Setup development environment

```
cd teams_bot_bridge
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2. Update main.py

- ☐ Import MS 365 Agents SDK modules
- ☐ Implement JWT validation for Azure Bot Service
- ☐ Add identity token generation for sa-tot-osa
- ☐ Implement message routing to ispirot-api
- ☐ Add session context management
- See: [../docs/TEAMS_SDK_INTEGRATION.md](#) → "Phase 2: Build Teams Bridge"

3. Create Dockerfile

- ☐ Copy or update from template
- ☐ Ensure all dependencies in requirements.txt

4. Create Teams app manifest (manifest.json)

- ☐ Configure app details
- ☐ Set scopes and capabilities
- See: [../docs/TEAMS_SDK_INTEGRATION.md](#) → "Teams App Manifest"

5. Test locally

- ☐ Run: `uvicorn main:app --reload`
- ☐ Health check: `curl http://localhost:8080/health`

- ☐ Review startup logs

Deliverables:

- ☐ main.py implementing MS 365 Agents SDK
- ☐ Updated requirements.txt with all dependencies
- ☐ Dockerfile (production-ready)
- ☐ manifest.json for Teams app
- ☐ .env.example with all variables
- ☐ Local testing confirmed working

Phase 3: Deploy to Cloud Run

Timeline: 1-2 hours

Owner: Development Team (GCP Admin for IAM)

Pre-Deployment Checks

- ☐ Verify all files are in place (main.py, Dockerfile, requirements.txt)
- ☐ Confirm ispirot-api is running and accessible
- ☐ Test identity token generation locally:

```
gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
  prod.iam.gserviceaccount.com
```

Deploy

1. Deploy to Cloud Run

```
cd teams_bot_bridge

PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="teams-ispilot-bridge"
SERVICE_ACCOUNT="sa-tot-osa@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud run deploy "${SERVICE_NAME}" \
  --source . \
  --region "${REGION}" \
  --project "${PROJECT_ID}" \
  --service-account "${SERVICE_ACCOUNT}" \
  --allow-unauthenticated \
  --memory 512Mi \
  --cpu 1 \
  --timeout 60 \
  --set-env-vars \
```

```
MICROSOFT_APP_ID=<APP_ID>,\nMICROSOFT_APP_PASSWORD=<APP_PASSWORD>,\nISPILOT_API_ENDPOINT=https://ispilot-api-46y2f3tyja-\nuc.a.run.app/chat,\nGOOGLE_CLOUD_PROJECT=${PROJECT_ID}
```

2. Get Service URL

```
gcloud run services describe teams-ispilot-bridge \n--region=us-central1 \n--project=corp-stro-salesinventory-prod \n--format='value(status.url)'
```

3. Verify Deployment

- ☐ Service is running: `gcloud run services describe teams-ispilot-bridge --region=us-central1`
- ☐ Health check: `curl https://<SERVICE-URL>/health`
- ☐ Check logs: `gcloud run logs read teams-ispilot-bridge --region=us-central1 --limit 20`

Deliverables:

- ☐ Cloud Run service deployed and running
- ☐ Service URL obtained
- ☐ Environment variables configured
- ☐ Health endpoint responding

Phase 4: Configure Azure & Teams

Timeline: 1-2 hours

Owner: Azure/Teams Admin + Development

Setup Azure Bot Service

- ☐ **Set Messaging Endpoint**
 - Go to Azure Bot Service → Configuration
 - Set Messaging Endpoint to: `https://<SERVICE-URL>/api/messages`
 - Save and verify
- ☐ **Test Bot in Emulator** (optional)
 - Download [Bot Framework Emulator](#)
 - Connect with Azure bot credentials
 - Send test message

Publish Teams App

- ☐ **Update manifest.json** with service URL and bot ID
- ☐ **Test in Teams Web Client**
 - Or publish to Teams App Catalog
- ☐ **Add app to team/group chat**
- ☐ **Send test message**
 - Expected: Bot responds with Reasoning Engine output

Deliverables:

- ☐ Azure messaging endpoint configured
 - ☐ Teams app manifest updated
 - ☐ App added to Teams test team/chat
 - ☐ End-to-end message flow verified
-

Phase 5: Validation & Go-Live

Timeline: 2-4 hours

Owner: Development Team + QA

Validation Checklist

Functional Testing

- ☐ Single-turn conversation works (send message, get response)
- ☐ Multi-turn conversation works (session continuity)
- ☐ Different message types handled (text, etc.)
- ☐ Error cases handled gracefully (timeouts, API errors)

Integration Testing

- ☐ Teams activity validation (correct JWT)
- ☐ Identity token generation works
- ☐ ispirot-api integration works end-to-end
- ☐ Session context persisted correctly

Performance Testing

- ☐ Response time < 30 seconds (typical)
- ☐ 100+ concurrent requests handled
- ☐ No memory leaks or resource exhaustion
- ☐ Cloud Run logs show healthy operation

Deployment Ready Checks

- ☐ All GitHub commits pushed
- ☐ Documentation updated
- ☐ Team trained on bot usage

- ☐ Support runbook created
- ☐ Monitoring/logging configured
- ☐ Rollback plan documented

Deliverables:

- ☐ All validation tests passing
- ☐ Performance benchmarks documented
- ☐ User documentation ready
- ☐ Support procedures established
- ☐ Ready for production rollout

Quick Reference

Key Files

File	Purpose	Status
main.py	Teams Bridge implementation	☒ Complete
requirements.txt	Python dependencies	☒ Complete
Dockerfile	Container definition	☒ Complete
manifest.json	Teams app manifest	☒ Complete
.env.example	Environment template	☒ Complete
deploy.sh	Cloud Run deployment	☒ Complete
README.md	Architecture & setup	☒ Complete
NEXT_STEPS.md	This file	☒ Complete

Critical Environment Variables

```
# Required for Phase 1
MICROSOFT_APP_ID=<from-azure-registration>
MICROSOFT_APP_PASSWORD=<from-azure-registration>

# Auto-configured
GOOGLE_CLOUD_PROJECT=corp-stro-salesinventory-prod
ISPILOT_API_ENDPOINT=https://ispilot-api-46y2f3tyja-uc.a.run.app/chat
```

Key API Endpoints

Endpoint	Method	Purpose
/health	GET	Health check

Endpoint	Method	Purpose
/api/messages	POST	Handle Teams activities
/api/activities	POST	Alternative handler (same)

Troubleshooting

Issue: "Identity token generation failed"

Solution: Verify sa-tot-osa service account exists and has Workload Identity bindings

Issue: "JWT validation failed"

Solution: Ensure MICROSOFT_APP_ID and MICROSOFT_APP_PASSWORD are correct from Azure

Issue: "ispilot-api returned 401"

Solution: Check identity token generation; verify sa-tot-osa has required permissions on ispilot-api

Issue: "No message text found in activity"

Solution: Verify Teams activity payload format; check TeamsActivityParser.extract_message() logic

Support Contacts

- **Azure/Teams Setup:** José Arturo (Phase 1)
- **Development:** Development Team (Phase 2)
- **Deployment:** GCP Admin (Phase 3)
- **Integration:** DevOps Team (Phase 4)
- **Testing/QA:** QA Team (Phase 5)

Success Criteria

☒ **Teams Bridge is ready when:**

- All phases completed
- All validation tests passing
- Response times meet SLA (< 30 seconds)
- Zero critical bugs
- Documentation complete
- Team trained and confident

Estimated Total Duration: 12-16 hours across all phases

Last Updated: 2026-09-01

Version: 1.0.0

Status: Ready for Phase 1 execution

- ☐ Cloud Run logs show no errors
- ☐ Monitor resource usage (CPU, memory)

Security Validation

- ☐ JWT validation rejects invalid tokens
- ☐ Identity tokens generated properly
- ☐ ispirot-api accepts only valid tokens
- ☐ No secrets in logs

Go-Live Readiness

- ☐ All tests passing
- ☐ Documentation updated
- ☐ Team trained on new interface
- ☐ Rollback plan documented
- ☐ Support team notified

Deliverables:

- ☐ Test report
- ☐ Performance baseline
- ☐ Runbook for support team
- ☐ Go/No-go decision

Timeline Overview

Phase	Duration	Status
Phase 1: Azure Setup	1-2 hrs	⌚ Waiting for José Arturo
Phase 2: Build Bridge	4-6 hrs	⌚ Blocked on Phase 1
Phase 3: Cloud Deploy	1-2 hrs	⌚ After Phase 2
Phase 4: Teams Config	1-2 hrs	⌚ After Phase 3
Phase 5: Validation	2-4 hrs	⌚ After Phase 4
Total	~12-16 hrs	-

Resources

Documentation

- [README.md](#) - Quick reference
- [../docs/TEAMS_SDK_INTEGRATION.md](#) - Complete implementation guide
- [../docs/ARCHITECTURE_RESET_TEAMS_SDK.md](#) - Architecture decisions
- [../ispilot-api/README.md](#) - ispirot-api reference

Code Templates

- main.py template in TEAMS_SDK_INTEGRATION.md
- Dockerfile in this directory

- [manifest.json](#) in this directory (needs update)

External Links

- [Microsoft 365 Agents SDK](#)
 - [Azure Bot Service](#)
 - [Teams App Manifest](#)
-

Questions?

Refer to:

1. [../docs/TEAMS_SDK_INTEGRATION.md](#) - Complete guide with detailed instructions
2. [README.md](#) - Troubleshooting section
3. [../WORK_CHECKPOINT.md](#) - Current implementation status